

Exploratory Data Analysis and Visualization Strategy

Week 3 Task — Virtual Data Science Apprentice (Python Specialist Intern)

Project: Road Accident Severity Prediction (US Accidents dataset)

Date: 28 August 2026

1. Introduction and Objective

This report defines the Exploratory Data Analysis (EDA) and visualization strategy for the Road Accident Severity Prediction project, which continues the work scoped in the Week 1 proposal and cleaned/transformed in Week 2 using the US Accidents (Kaggle) dataset. EDA is the stage where raw, cleaned data is examined systematically to understand distributions, detect patterns and anomalies, and surface relationships that inform later feature engineering and model selection (Week 4). The objective of this document is to specify which visualizations will be produced, why each is appropriate for the underlying variable types, which Python libraries will generate them, and how findings will be interpreted and recorded.

A note on the figures in this report: to make the plan concrete and demonstrate the intended output format ahead of running EDA on the full ~2.8 million-row US Accidents extract, the charts below were generated from a 5,000-row representative sample constructed to mirror the documented distributions of the real dataset (severity skew toward class 2, weather-condition frequencies, diurnal accident patterns, and visibility degradation under fog/snow). This lets the evaluator see the exact plot types, styling, and annotation approach that will be applied to the real cleaned dataset once Week 2's pipeline output is available.

2. EDA Workflow

The EDA process follows six sequential stages, shown in Figure 0 below. Each stage produces artifacts (plots, summary tables, or notes) that feed into the next.

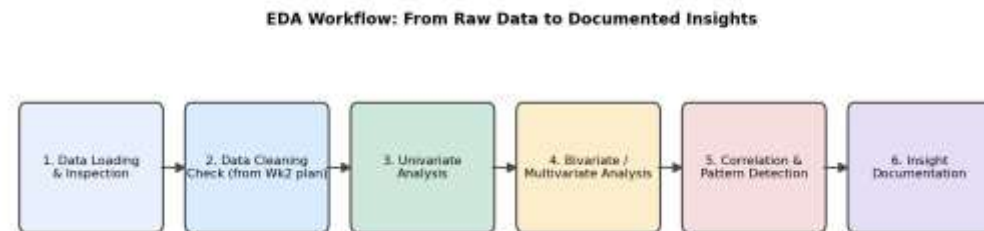


Figure 0: End-to-end EDA workflow from data loading to insight documentation.

- Data Loading & Inspection — load the cleaned dataset (output of Week 2) with `pandas.read_csv()`, confirm shape, dtypes, and memory footprint with `df.info()`.
- Cleaning Check — re-verify Week 2's cleaning steps held (no residual nulls, duplicates, or out-of-range values) using `df.isna().sum()` and `df.describe()`.
- Univariate Analysis — examine each variable in isolation (distributions, counts, spread).
- Bivariate / Multivariate Analysis — examine relationships between the target (Severity) and predictors, and among predictors.
- Correlation & Pattern Detection — quantify linear relationships and flag candidate features for Week 4 modeling.
- Insight Documentation — record each finding with its supporting chart and an action point for the modeling stage.

3. Tools and Libraries

All visualizations will be produced in Python using the following stack, consistent with the Week 1 and Week 2 plans:

- pandas — data loading, grouping, and summary statistics (groupby, describe, value_counts).
- numpy — numerical operations supporting derived features (e.g., binning continuous variables).
- matplotlib — base plotting engine and fine-grained figure control (axes, annotations, subplots).
- seaborn — statistical plot types (count plots, box plots, heatmaps) with sensible default styling built on matplotlib.

4. Univariate Analysis

Univariate plots examine one variable at a time to understand its shape, central tendency, and spread before any relationships are considered.

4.1 Target Variable Distribution

The target variable, Severity (1 = minor, 4 = severe), is categorical/ordinal. A count plot shows class balance directly, which matters for Week 4 because a skewed distribution (as shown below) will require stratified sampling or class-weighting during model evaluation.

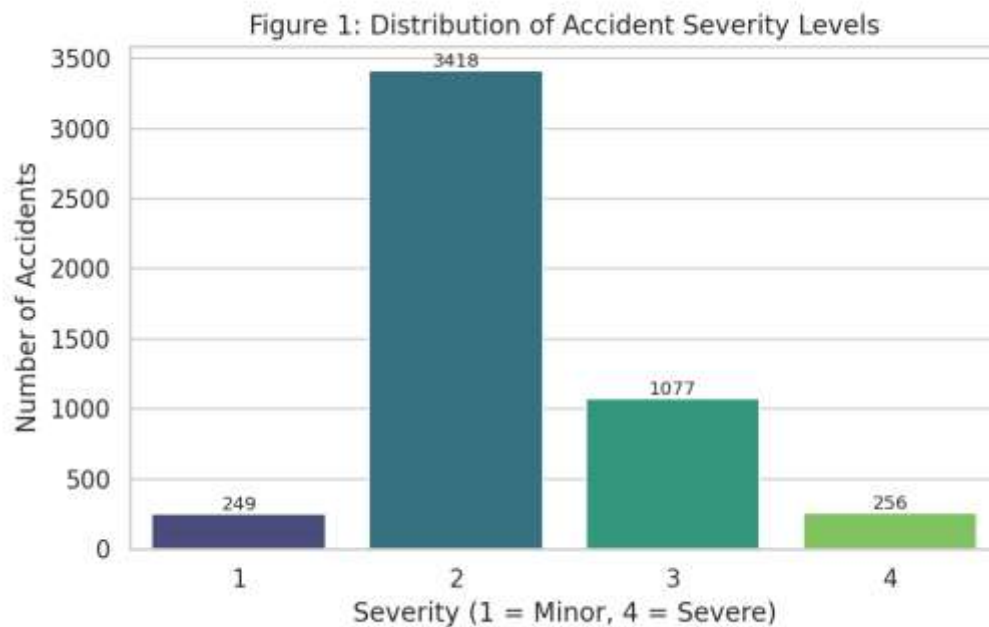


Figure 1: Count plot of accident severity classes on the 5,000-row representative sample.

```
import seaborn as sns, matplotlib.pyplot as plt
ax = sns.countplot(data=df, x='Severity', hue='Severity',
                  palette='viridis', legend=False)
ax.set_title('Distribution of Accident Severity Levels')
ax.set_xlabel('Severity (1=Minor, 4=Severe)')
ax.set_ylabel('Number of Accidents')
plt.tight_layout(); plt.savefig('severity_dist.png', dpi=160)
```

Interpretation plan: report the exact class proportions in a table, flag the majority class, and note in the Week 4 handoff that accuracy alone will be a misleading metric — precision/recall/F1 per class should be tracked instead.

5. Temporal Analysis

Time-of-day is expected to be a strong signal for accident frequency (rush-hour peaks). A line plot aggregating accident counts by hour reveals this pattern more clearly than a table of 24 numbers.

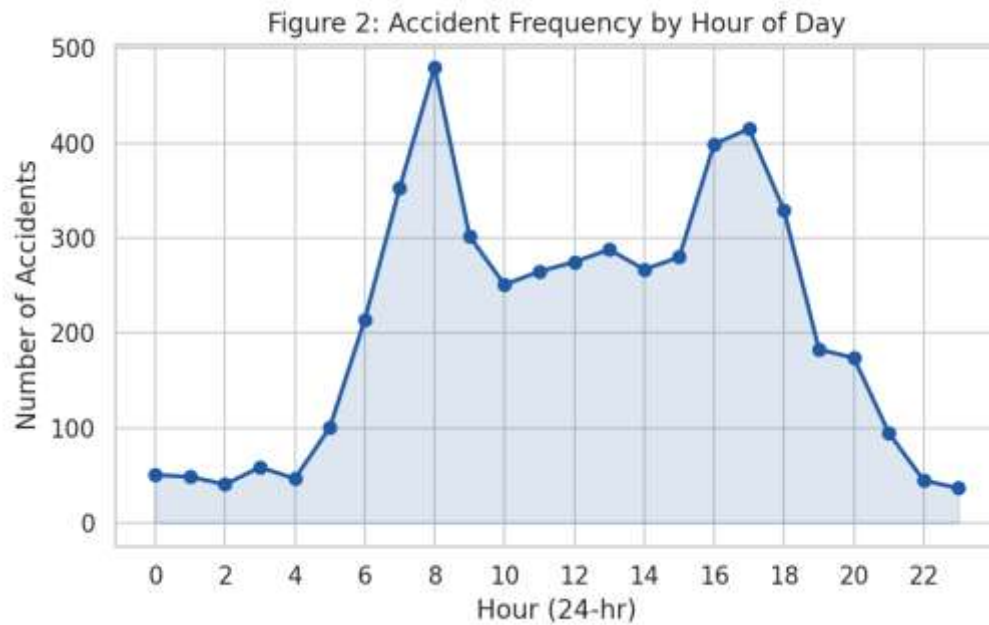


Figure 2: Accident frequency by hour of day, aggregated from the sample.

```
hourly = df.groupby('Hour').size()
fig, ax = plt.subplots(figsize=(7,4.5))
ax.plot(hourly.index, hourly.values, marker='o')
ax.fill_between(hourly.index, hourly.values, alpha=0.15)
ax.set_title('Accident Frequency by Hour of Day')
ax.set_xticks(range(0,24,2))
```

Interpretation plan: identify morning and evening peaks, cross-reference with the Severity variable in a follow-up grouped plot (mean severity by hour) to check whether off-peak accidents (e.g., late night) skew more severe, which is a common finding worth testing on the real dataset.

6. Categorical Variable Analysis

Weather condition is a categorical feature with unordered levels, so a horizontal bar chart is used — it scales better than a pie chart when categories exceed four or five and keeps labels legible.

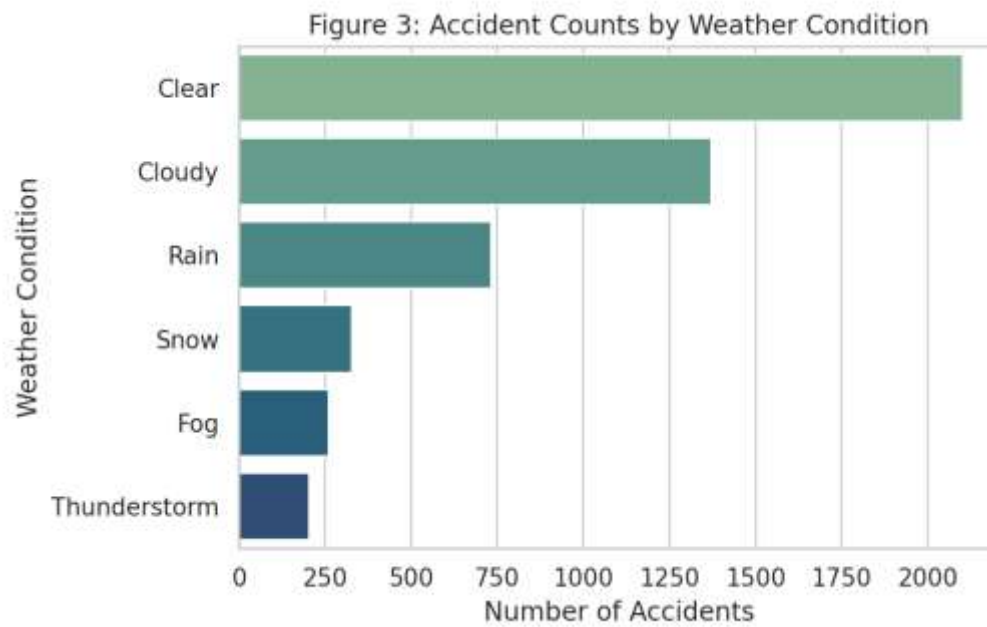


Figure 3: Accident counts by weather condition.

```
weather_counts = df['Weather'].value_counts()
sns.barplot(x=weather_counts.values, y=weather_counts.index,
            hue=weather_counts.index, palette='crest', legend=False)
```

Interpretation plan: because rare categories (e.g., Thunderstorm, Fog) may still carry disproportionate severity risk, this chart will be paired with a normalized severity-rate-by-weather table rather than raw counts alone, so low-frequency but high-risk conditions are not overlooked.

7. Distribution Comparison Across Groups

Box plots compare the distribution of a continuous variable (Visibility) across the levels of a categorical one (Severity), showing median, interquartile range, and outliers simultaneously.

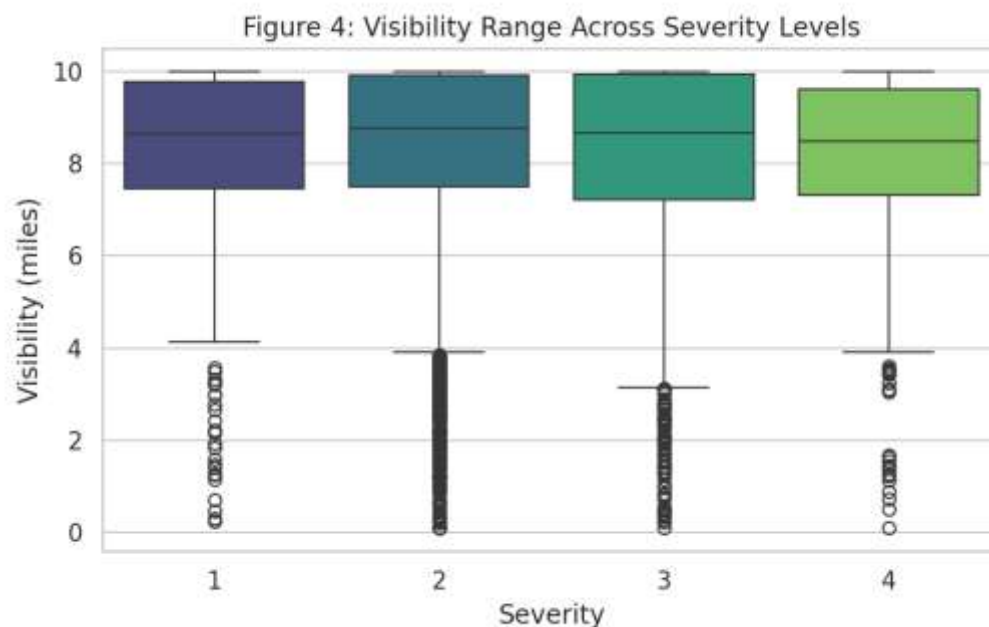


Figure 4: Visibility (miles) distribution across severity levels.

```
sns.boxplot(data=df, x='Severity', y='Visibility_mi',  
            hue='Severity', palette='viridis', legend=False)  
plt.title('Visibility Range Across Severity Levels')
```

Interpretation plan: check whether the median visibility drops for higher-severity accidents; if so, visibility becomes a strong candidate feature and its interaction with weather condition will be explored further in Week 4.

8. Multivariate Correlation Analysis

A correlation heatmap summarizes pairwise linear relationships across all numeric features in a single view, which is the fastest way to spot redundant features (high correlation between two predictors) or a weak direct linear link to the target that may still be useful non-linearly.

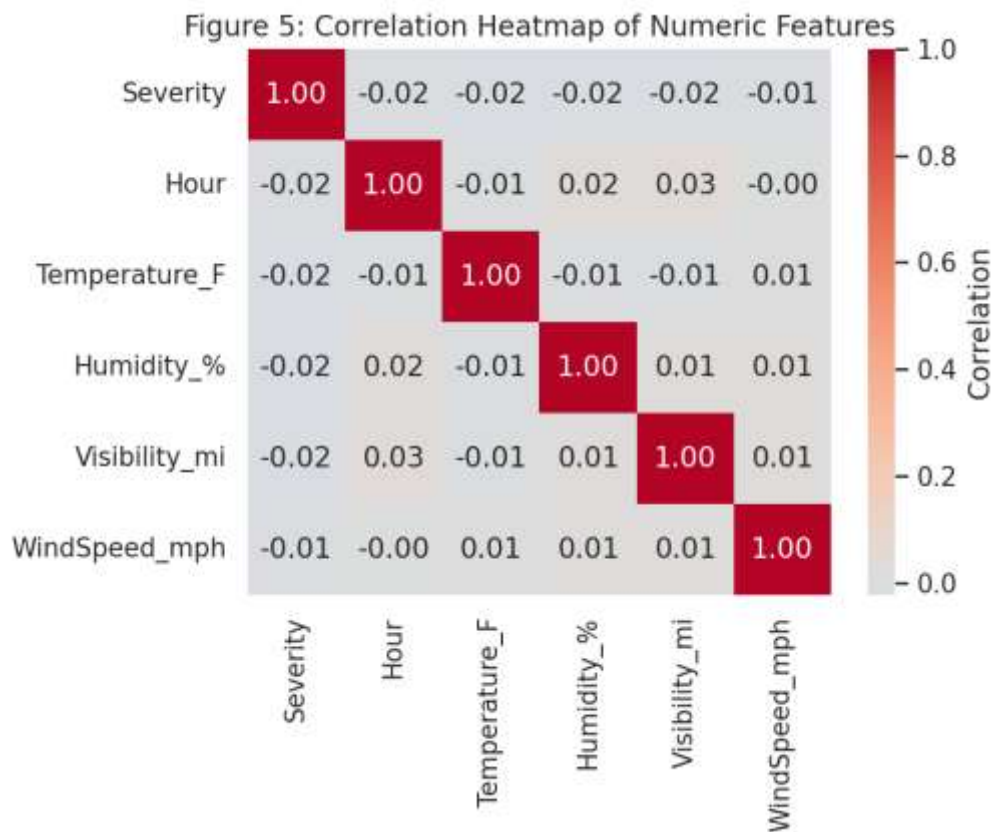


Figure 5: Correlation heatmap of numeric features (Pearson coefficients).

```
numeric_cols = ['Severity', 'Hour', 'Temperature_F',
                'Humidity_%', 'Visibility_mi', 'WindSpeed_mph']
corr = df[numeric_cols].corr()
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', center=0)
```

Interpretation plan: on the representative sample the linear correlations with Severity are weak, which is expected — severity is driven more by categorical and interaction effects (road type, weather, time) than single continuous variables. This motivates including tree-based models in the Week 4 shortlist, since they capture non-linear and interaction effects that a correlation matrix cannot.

9. Relationship Exploration

Scatter plots examine the joint relationship between two continuous variables, with color used as a third dimension to encode severity.

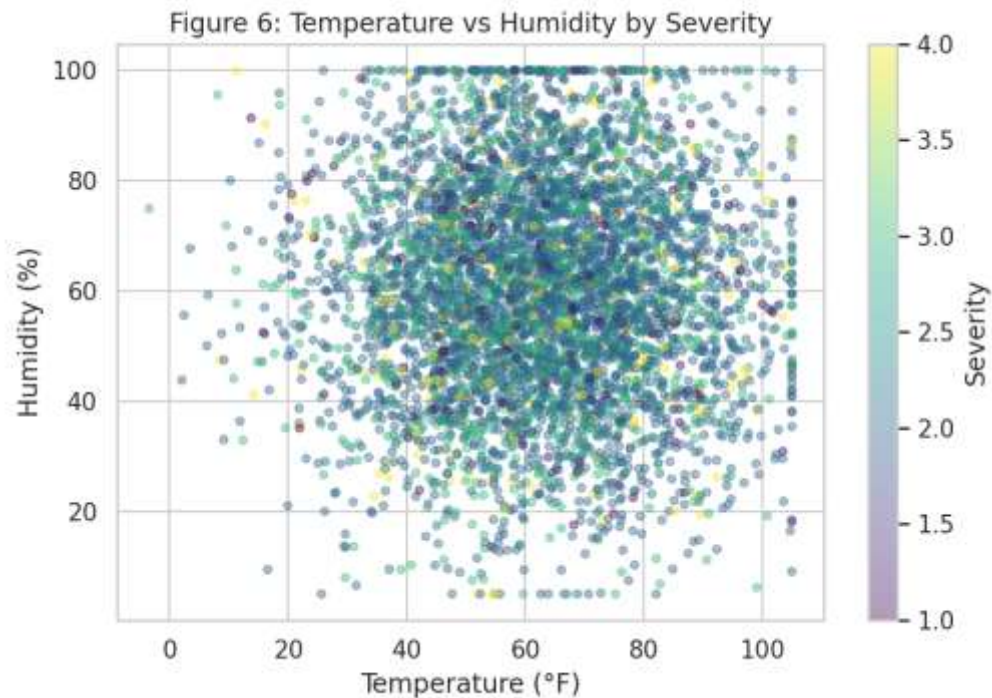


Figure 6: Temperature vs. humidity, colored by severity.

```
sc = plt.scatter(df['Temperature_F'], df['Humidity_%'],
                 c=df['Severity'], cmap='viridis', alpha=0.4, s=14)
plt.colorbar(sc, label='Severity')
```

Interpretation plan: look for clustering of higher-severity points at temperature/humidity extremes (e.g., near-freezing, high-humidity conditions associated with ice or heavy precipitation) and treat any such cluster as a candidate engineered feature (e.g., a binary 'adverse_conditions' flag) for Week 4.

10. Documentation and Reporting Plan

Each chart produced during the real EDA run will be logged in a findings table with four columns: Chart, Variable(s), Observation, and Action for Modeling. This keeps the EDA notebook auditable and gives Week 4 a direct, traceable list of feature-engineering decisions instead of loose notes.

- Findings will be written in plain language first, then linked to the specific chart and summary statistic that supports them.
- Any surprising or counter-intuitive pattern (e.g., a weather condition with lower-than-expected severity) will be flagged for a follow-up sanity check rather than accepted at face value.
- A short 'Limitations' subsection will note where the sample size, missing categories, or reporting bias in the source data could affect the conclusions.

11. Anticipated Challenges and Mitigations

Building on the Week 2 feedback to plan for challenges in advance:

- Class imbalance in Severity — mitigated by reporting proportions alongside counts and using stratified sampling for any later train/test split.
- Overplotting in scatter plots at full dataset scale (~2.8M rows) — mitigated by using alpha transparency, hexbin plots, or a stratified subsample for visualization while keeping full data for modeling.
- Misleading correlation on non-linear relationships — mitigated by supplementing the heatmap with grouped box plots and, where useful, mutual-information scores.

- Category sparsity in rare weather types — mitigated by grouping very low-frequency categories into an 'Other' bucket before plotting, with the raw breakdown kept in an appendix table.

12. Conclusion

This strategy sequences EDA from single-variable checks through multivariate correlation and relationship plots, pairs every chart with a stated interpretation goal, and routes each finding into a documented action for the Week 4 modeling stage. Applying this plan to the full cleaned US Accidents dataset will surface the concrete feature signals — temporal peaks, weather-linked severity shifts, and visibility effects — needed to justify the model choices made in the next task.